

Сети Петри

Дисциплина: Имитационное моделирование

Верниковская Екатерина Андреевна

Содержание

1	Реализация основных моделей в агентном подходе	5
1.1	Сети Петри	5
1.2	Обедающие философы (задача Дейкстры)	5
1.3	Основные скрипты	7
1.4	Код модели	7
1.5	Базовые эксперименты	17
1.6	Анимация процесса (классическая сеть)	21
1.7	Анимация процесса (сеть с арбитром)	24
1.8	Итоговый отчёт	27

Список иллюстраций

1.1	Обедающие философы	6
-----	------------------------------	---

Список таблиц

1 Реализация основных моделей в агентном подходе

1.1 Сети Петри

Сеть Петри - математический аппарат для моделирования дискретных систем с параллелизмом, асинхронностью и синхронизацией. Графически это двудольный ориентированный граф, содержащий два типа вершин:

- **Позиции (places)** - пассивные элементы (кружки), описывающие состояние системы. Внутри могут находиться фишки (tokens)
- **Переходы (transitions)** - активные элементы (прямоугольники), описывающие события

Правило срабатывания: переход разрешён, если во всех его входных позициях есть нужное количество фишек. При срабатывании фишки из входных позиций удаляются и добавляются в выходные

Ключевые свойства: ограниченность, активность (живость), достижимость, сохраняемость

1.2 Обедающие философы (задача Дейкстры)

Постановка: N философов сидят за круглым столом. У каждого своя тарелка, между соседями - одна вилка. Чтобы поесть, нужны две вилки (левая и правая)

Проблема: при наивной реализации (каждый сначала берёт левую вилку) возникает **взаимная блокировка (deadlock)** - все философы держат по одной вилке и ждут вторую.

Решение (в работе): добавление арбитра - дополнительной позиции с N-1 фишками, ограничивающей количество одновременно едящих философов

Моделирование: каждый философ имеет состояния «Думает» → «Голоден» → «Ест». Переходы: взять левую вилку, взять правую вилку, положить вилки

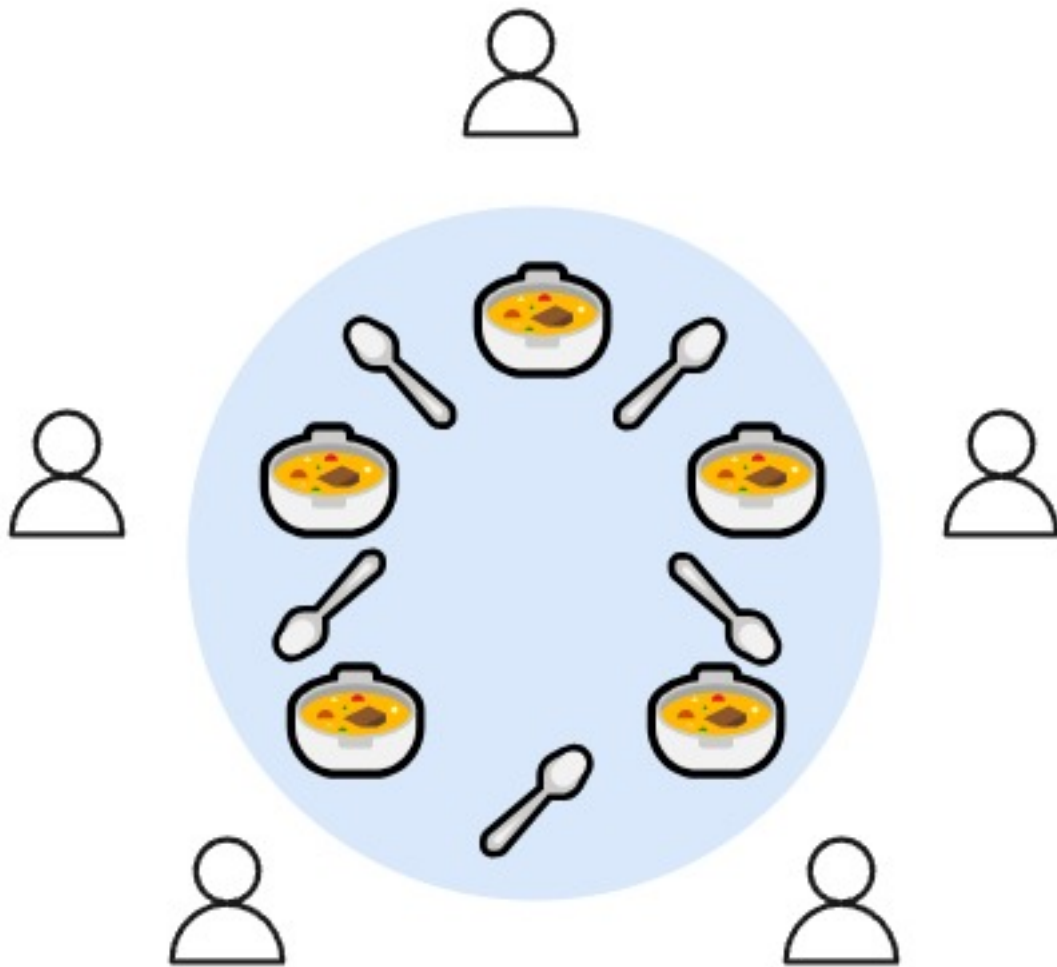


Рисунок 1.1: Обедающие философы

1.3 Основные скрипты

`src/DiningPhilosophers.jl` - модуль с определением сети Петри, построением классической модели и модели с арбитром, функциями симуляции (ODE, стохастика), обнаружения deadlock и визуализации

`scripts/dining_philosophers.jl` - запуск стохастического моделирования для обеих моделей, сохранение CSV и графиков, вывод факта deadlock

`scripts/dining_philosophers_animation_classic.jl` - создание GIF-анимации изменения маркировки сети Петри во времени для классической сети

`scripts/dining_philosophers_animation_arbiter.jl` - создание GIF-анимации изменения маркировки сети Петри во времени для сети с арбитром

`scripts/dining_philosophers_report.jl` - построение итогового сравнительного графика числа «едящих» философов для классической модели и модели с арбитром

1.4 Код модели

```
module DiningPhilosophers
```

```
using OrdinaryDiffEq
```

```
using Plots
```

```
using DataFrames
```

```
using Random, LinearAlgebra
```

```
export build_classical_network, build_arbiter_network
```

```
export simulate_ode, simulate_stochastic
```

```
export detect_deadlock, plot_marking_evolution
```

```
### Определение простой структуры PetriNet
```

```
struct PetriNet
```

```
    n_places::Int
```

```
    n_transitions::Int
```

```
    incidence::Matrix{Int}
```

```
    place_names::Vector{Symbol}
```

```
    transition_names::Vector{Symbol}
```

```
end
```

```
function PetriNet(
```

```
    n_places,
```

```
    n_transitions;
```

```
    place_names = Symbol[],
```

```
    transition_names = Symbol[],
```

```
)
```

```
    incidence = zeros{Int, n_places, n_transitions}
```

```
    if isempty(place_names)
```

```
        place_names = [Symbol("p$i") for i = 1:n_places]
```

```
    end
```

```
    if isempty(transition_names)
```

```
        transition_names = [Symbol("t$i") for i = 1:n_transitions]
```

```
    end
```

```
    PetriNet(n_places, n_transitions, incidence, place_names,
```

```
        transition_names)
```

```
end
```



```

function add_arc!(net::PetriNet, place::Int, transition::Int,
    ↪ sign::Int)
    net.incidence[place, transition] += sign
end

```

Построение сетей Петри

```

function build_classical_network(N::Int)
    n_places = 4N
    n_transitions = 3N
    net = PetriNet(n_places, n_transitions)

    for i = 1:N
        net.place_names[i] = Symbol("Think_$i")
        net.place_names[N+i] = Symbol("Hungry_$i")
        net.place_names[2N+i] = Symbol("Eat_$i")
        net.place_names[3N+i] = Symbol("Fork_$i")
    end

    for i = 1:N
        net.transition_names[i] = Symbol("GetLeft_$i")
        net.transition_names[N+i] = Symbol("GetRight_$i")
        net.transition_names[2N+i] = Symbol("PutForks_$i")
    end

    for i = 1:N
        think = i
        hungry = N + i
        eat = 2N + i
    end
end

```

```

left_fork = 3N + i
right_fork = 3N + (i % N + 1)

get_left = i
get_right = N + i
put_forks = 2N + i

add_arc!(net, think, get_left, -1)
add_arc!(net, left_fork, get_left, -1)
add_arc!(net, hungry, get_left, +1)

add_arc!(net, hungry, get_right, -1)
add_arc!(net, right_fork, get_right, -1)
add_arc!(net, eat, get_right, +1)

add_arc!(net, eat, put_forks, -1)
add_arc!(net, think, put_forks, +1)
add_arc!(net, left_fork, put_forks, +1)
add_arc!(net, right_fork, put_forks, +1)
end

u0 = zeros(Float64, n_places)

for i = 1:N
    u0[i] = 1.0 # Think_i
    u0[3N+i] = 1.0 # Fork_i
end

return net, u0, net.place_names

```

end

```
function build_arbiter_network(N::Int)
```

```
    n_places = 4N + 1
```

```
    n_transitions = 3N
```

```
    net = PetriNet(n_places, n_transitions)
```

```
    for i = 1:N
```

```
        net.place_names[i] = Symbol("Think_$i")
```

```
        net.place_names[N+i] = Symbol("Hungry_$i")
```

```
        net.place_names[2N+i] = Symbol("Eat_$i")
```

```
        net.place_names[3N+i] = Symbol("Fork_$i")
```

```
    end
```

```
    net.place_names[4N+1] = :Arbiter
```

```
    for i = 1:N
```

```
        net.transition_names[i] = Symbol("GetLeft_$i")
```

```
        net.transition_names[N+i] = Symbol("GetRight_$i")
```

```
        net.transition_names[2N+i] = Symbol("PutForks_$i")
```

```
    end
```

```
    arbiter_idx = 4N + 1
```

```
    for i = 1:N
```

```
        think = i
```

```
        hungry = N + i
```

```
        eat = 2N + i
```

```
        left_fork = 3N + i
```

```

right_fork = 3N + (i % N + 1)
get_left = i
get_right = N + i
put_forks = 2N + i
add_arc!(net, think, get_left, -1)
add_arc!(net, left_fork, get_left, -1)
add_arc!(net, arbiter_idx, get_left, -1)
add_arc!(net, hungry, get_left, +1)
add_arc!(net, hungry, get_right, -1)
add_arc!(net, right_fork, get_right, -1)
add_arc!(net, eat, get_right, +1)
add_arc!(net, eat, put_forks, -1)
add_arc!(net, think, put_forks, +1)
add_arc!(net, left_fork, put_forks, +1)
add_arc!(net, right_fork, put_forks, +1)
add_arc!(net, arbiter_idx, put_forks, +1)
end

u0 = zeros(Float64, n_places)
for i = 1:N
    u0[i] = 1.0
    u0[3N+i] = 1.0
end

u0[arbiter_idx] = N - 1
return net, u0, net.place_names
end

```

Детерминированное моделирование (ODE)

```

function vectorfield(net::PetriNet, rates =
↳ ones(net.n_transitions))
    function f!(du, u, params, t)
        a = zeros(net.n_transitions)
        for j = 1:net.n_transitions
            rate = rates[j]
            prod = rate
            for i = 1:net.n_places
                if net.incidence[i, j] < 0
                    prod *= u[i] ^ (-net.incidence[i, j])
                end
            end
            a[j] = prod
        end
        du .= net.incidence * a
    end
    return f!
end

function simulate_ode(net::PetriNet, u0::Vector{Float64},
↳ tmax::Float64; saveat = 0.1)
    f = vectorfield(net)
    prob = ODEProblem(f, u0, (0.0, tmax))
    sol = solve(prob, Tsit5(), saveat = saveat)
    df = DataFrame(time = sol.t)
    for i = 1:net.n_places
        df[!, String(net.place_names[i])] = sol[i, :]
    end
    return df
end

```

end

Стохастическое моделирование (алгоритм Гиллеспи)

```
function simulate_stochastic(  
    net::PetriNet,  
    u0::Vector{Float64},  
    tmax::Float64;  
    rates = ones(net.n_transitions),  
    rng = Random.GLOBAL_RNG,  
)  
    u = copy(u0)  
    t = 0.0  
    times = [t]  
    states = [copy(u)]  
  
    while t < tmax  
        a = zeros(net.n_transitions)  
        for j = 1:net.n_transitions  
            rate = rates[j]  
            prod = rate  
            for i = 1:net.n_places  
                if net.incidence[i, j] > 0  
                    prod *= u[i] ^ (-net.incidence[i, j])  
                end  
            end  
            a[j] = prod  
        end  
        a0 = sum(a)  
        if a0 == 0  
            continue  
        end  
        j = rand(1:length(a), rng)  
        t = t + 1/a0  
        times = [times; t]  
        u = copy(u)  
        for i = 1:net.n_places  
            u[i] = u[i] - net.incidence[i, j]  
        end  
        for i = 1:net.n_places  
            u[i] = u[i] + net.output[i, j]  
        end  
        states = [states; copy(u)]  
    end  
end
```

```

        break
    end
    dt = -log(rand(rng)) / a0
    r = rand(rng) * a0
    cumsum = 0.0
    chosen = 1
    for j = 1:net.n_transitions
        cumsum += a[j]
        if r <= cumsum
            chosen = j
            break
        end
    end
    for i = 1:net.n_places
        u[i] += net.incidence[i, chosen]
    end
    t += dt
    if t <= tmax
        push!(times, t)
        push!(states, copy(u))
    end
end
df = DataFrame(time = times)
for i = 1:net.n_places
    df[!, String(net.place_names[i])] = [s[i] for s in states]
end
return df
end

```

Обнаружение deadlock

```
function detect_deadlock(df::DataFrame, net::PetriNet; tol = 1e-6)
    u_last = [df[end, String(net.place_names[i])] for i =
        ↪ 1:net.n_places]
    for j = 1:net.n_transitions
        can_fire = true
        for i = 1:net.n_places
            if net.incidence[i, j] < 0 && u_last[i] <
                ↪ -net.incidence[i, j] - tol
                can_fire = false
                break
            end
        end
        if can_fire
            return false
        end
    end
    return true
end
```

Визуализация

```
function plot_marking_evolution(df::DataFrame, N::Int)
    plots = []
    for group in ["Think", "Hungry", "Eat", "Fork"]
        p = plot(xlabel = "Time", ylabel = group, title = "$group
            ↪ states")
        for i = 1:N
            col = "$(group)_$i"
            if col in names(df)
```



```

        plot!(df.time, df[!, col], label = "$(group)_{i}")
    end
end
push!(plots, p)
end
return plot(plots..., layout = (4, 1), size = (800, 1000))
end

end # module

```

Main.DiningPhilosophers

1.5 Базовые эксперименты

using DataFrames, CSV, Plots

default(fmt = :png)

N = 5

tmax = 50.0

```

println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))

```

```
println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))

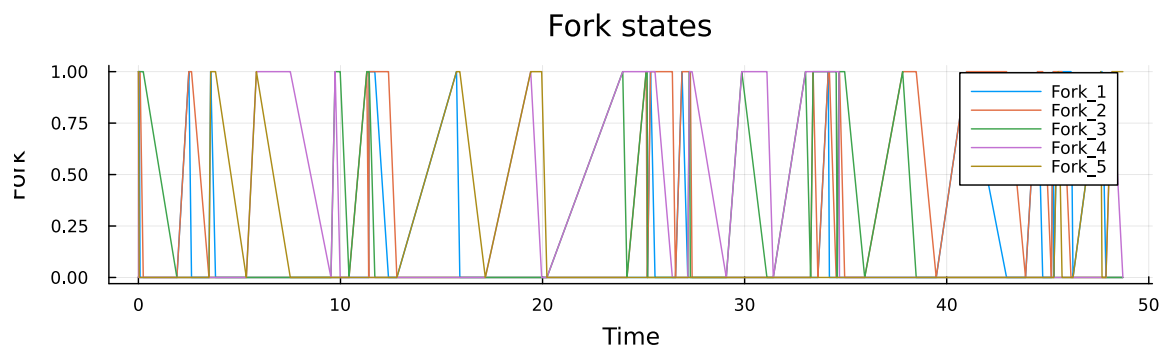
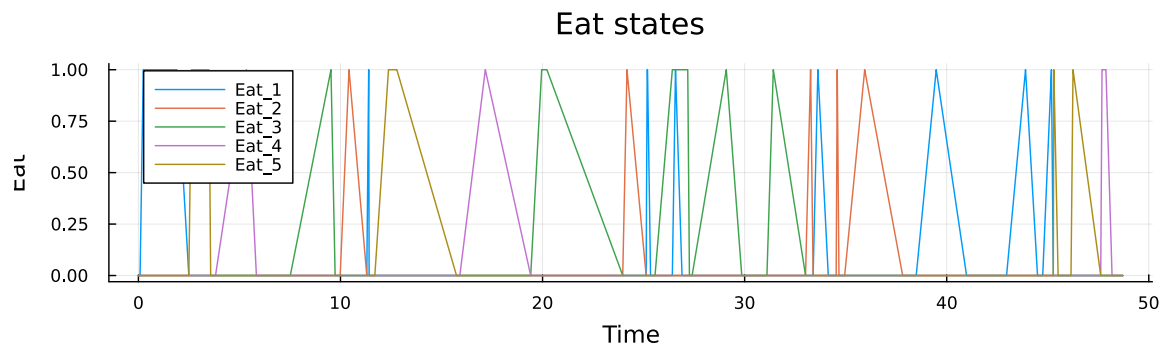
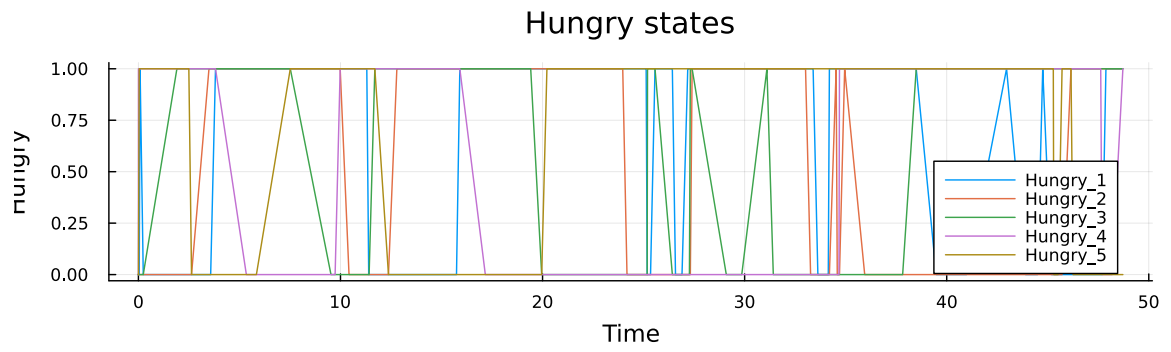
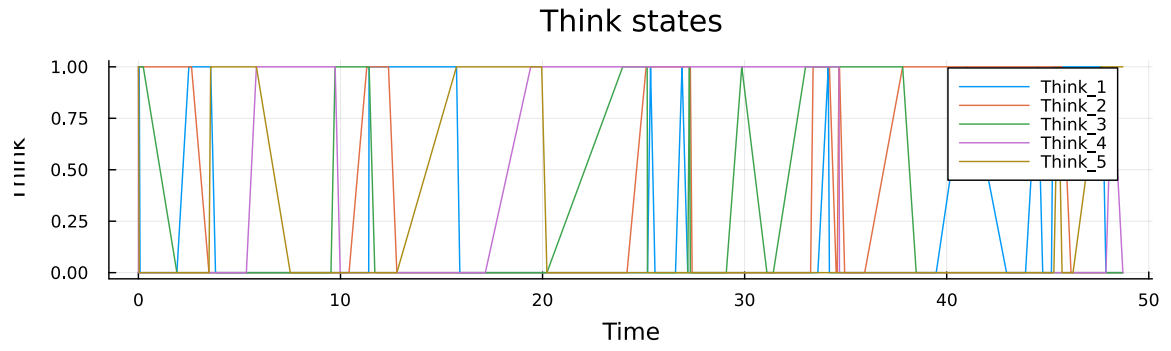
display(plot_classic)
display(plot_arb)
```

=== Классическая сеть (без арбитра) ===

Deadlock обнаружен: true

=== Сеть с арбитром ===

Deadlock обнаружен: false



1.6 Анимация процесса (классическая сеть)

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate"project"
using DrWatson
@quickactivate "project"
using Plots, Random

N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)

Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
    bar(
        1:length(u),
        u,
        legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "Позиция",
        ylabel = "Фишки",
        title = "Время = $(round(row.time, digits=2))",
    )
    xticks!(1:length(u), string.(names), rotation = 45)
```

end

```
gif(anim, plotsdir("philosophers_simulation_classic.gif"), fps =  
    ↪ 2)  
println("Анимация сохранена в  
    ↪ plots/philosophers_simulation_classic.gif")
```

```
Activating project at `~/work/study/2025-2026/Имитационное моделирование/simu  
modeling/labs/lab05/project`
```

```
└ Warning: DrWatson could not find find a project file by recursively checking  
  | (given dir: /home/eavernikovskaya)  
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84  
└ Warning: DrWatson could not find find a project file by recursively checking  
  | (given dir: /home/eavernikovskaya)  
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

Анимация сохранена в plots/philosophers_simulation_classic.gif

```
[ Info: Saved animation to /home/eavernikovskaya/work/study/2025-  
2026/Имитационное моделирование/simulation-modeling/labs/lab05/project/plots/ph
```

Сохраняем последний кадр как статическое изображение

```
last_row = df[end, :]  
u_last = [last_row[col] for col in propertynames(last_row) if col  
    ↪ != :time]  
p = bar(  
    1:length(u_last),  
    u_last,  
    legend = false,
```

```

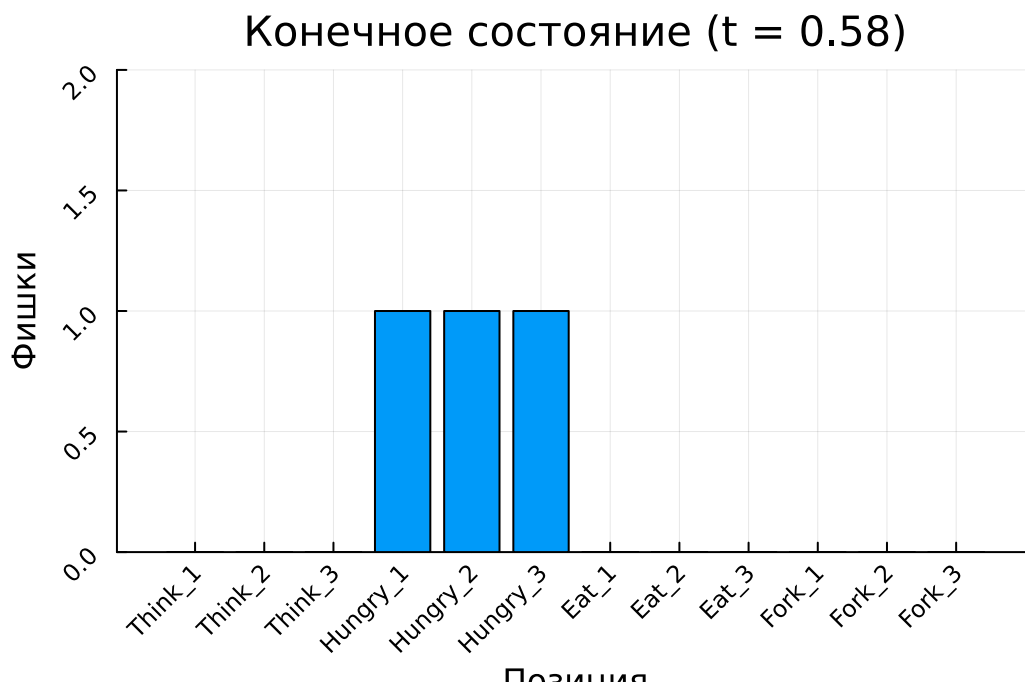
ylims = (0, maximum(u0) + 1),
xlabel = "Позиция",
ylabel = "Фишки",
title = "Конечное состояние (t = $(round(last_row.time,
↪ digits=2)))",
)
xticks!(1:length(u_last), string.(names), rotation = 45)
savefig(plotsdir("philosophers_simulation_classic_frame.png"))
println("Кадр сохранён в
↪ plots/philosophers_simulation_classic_frame.png")

```

Кадр сохранён в plots/philosophers_simulation_classic_frame.png

Отображаем последний кадр

р



1.7 Анимация процесса (сеть с арбитром)

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate "project"
using DrWatson
@quickactivate "project"
using Plots, Random

N = 3
tmax = 30.0
net, u0, names = build_arbiter_network(N)

Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
    bar(
        1:length(u),
        u,
        legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "Позиция",
        ylabel = "Фишки",
        title = "Время = $(round(row.time, digits=2))",
    )
    xticks!(1:length(u), string.(names), rotation = 45)
```


end

```
gif(anim, plotsdir("philosophers_simulation_arbiter.gif"), fps =  
    ↪ 2)  
println("Анимация сохранена в  
    ↪ plots/philosophers_simulation_arbiter.gif")
```

```
Activating project at `~/work/study/2025-2026/Имитационное моделирование/simu  
modeling/labs/lab05/project`
```

```
└ Warning: DrWatson could not find find a project file by recursively checking  
  | (given dir: /home/eavernikovskaya)  
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84  
└ Warning: DrWatson could not find find a project file by recursively checking  
  | (given dir: /home/eavernikovskaya)  
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

Анимация сохранена в plots/philosophers_simulation_arbiter.gif

```
[ Info: Saved animation to /home/eavernikovskaya/work/study/2025-  
2026/Имитационное моделирование/simulation-modeling/labs/lab05/project/plots/ph
```

Сохраняем последний кадр как статическое изображение

```
last_row = df[end, :]  
u_last = [last_row[col] for col in propertynames(last_row) if col  
    ↪ != :time]  
p = bar(  
    1:length(u_last),  
    u_last,  
    legend = false,
```

```

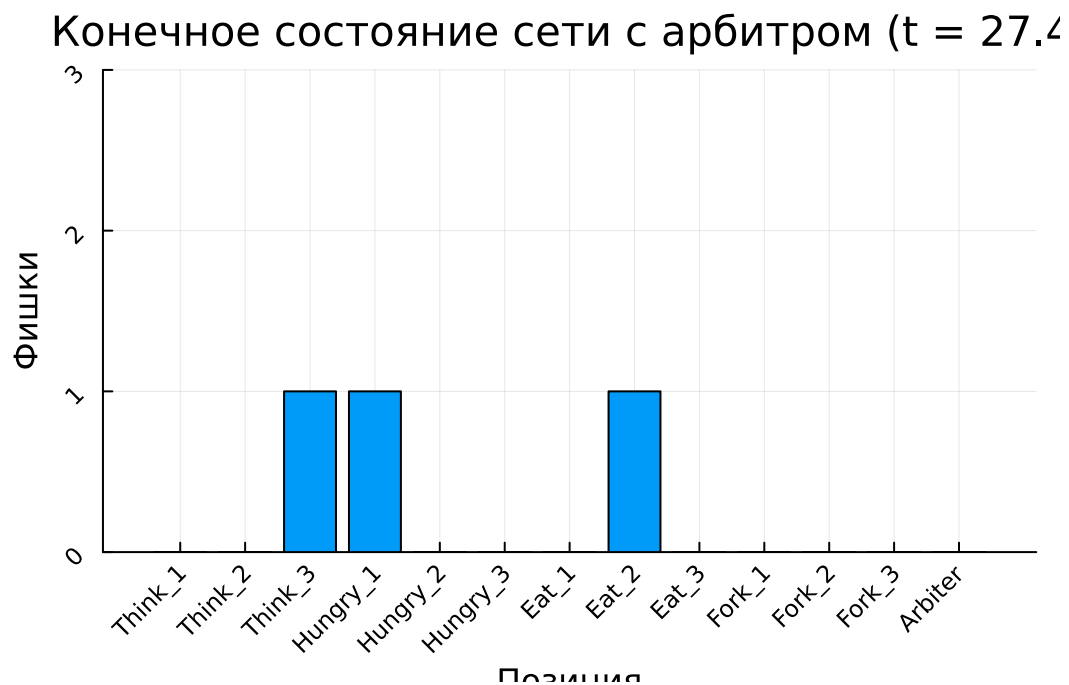
ylims = (0, maximum(u0) + 1),
xlabel = "Позиция",
ylabel = "Фишки",
title = "Конечное состояние сети с арбитром (t =
↪ $(round(last_row.time, digits=2)))",
)
xticks!(1:length(u_last), string.(names), rotation = 45)
savefig(plotsdir("philosophers_simulation_arbiter_frame.png"))
println("Кадр сохранён в
↪ plots/philosophers_simulation_arbiter_frame.png")

```

Кадр сохранён в plots/philosophers_simulation_arbiter_frame.png

Отображаем последний кадр

р



1.8 Итоговый отчёт

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate"project"
using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots
default(fmt = :png)
```

```
df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
N = 5
```

```
Activating project at `~/work/study/2025-2026/Имитационное моделирование/simu
modeling/labs/lab05/project`
```

```
└ Warning: DrWatson could not find find a project file by recursively checking
  | (given dir: /home/eavernikovskaya)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
└ Warning: DrWatson could not find find a project file by recursively checking
  | (given dir: /home/eavernikovskaya)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

5

Столбцы для состояния «Ест»

```
eat_cols = [Symbol("Eat_$i") for i = 1:N]
```

```

p1 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = [" $\phi$   $i$ " for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест ( $1/\theta$ )",
    title = "Классическая сеть",
)

p2 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[:, eat_cols]),
    label = [" $\phi$   $i$ " for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест ( $1/\theta$ )",
    title = "Сеть с арбитром",
)

p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotsdir("final_report.png"))
println("Отчёт сохранён в plots/final_report.png")

display(p_final)

```

Отчёт сохранён в plots/final_report.png

